

임베디드 소프트웨어

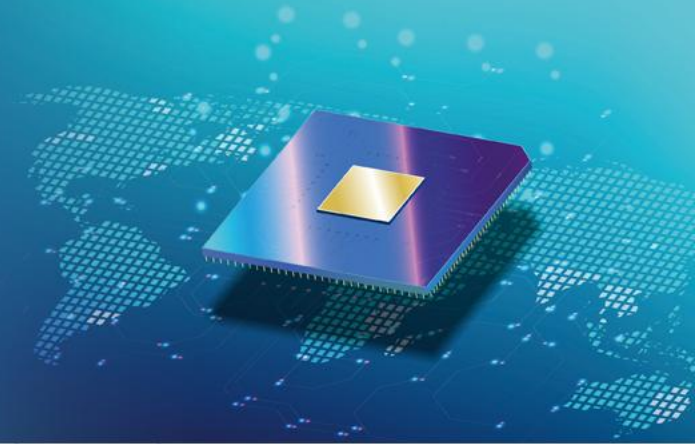
서울대학교 차세대반도체 혁신융합대학





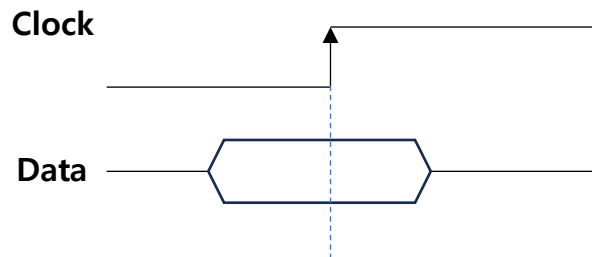
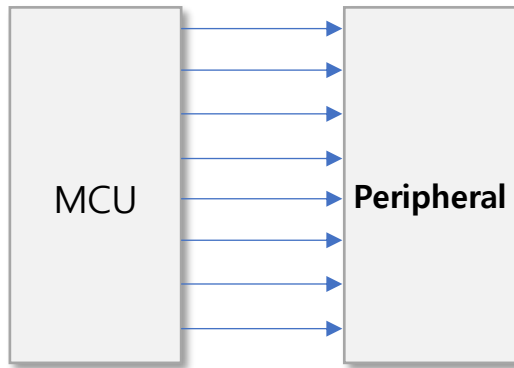
CHAPTER 6

SPI와 MAX7219



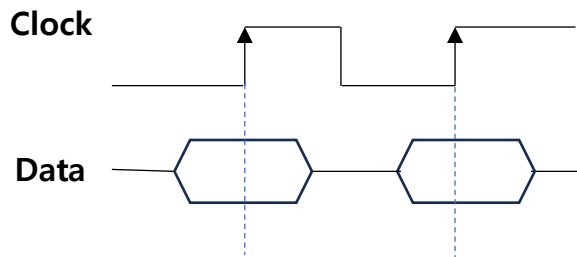
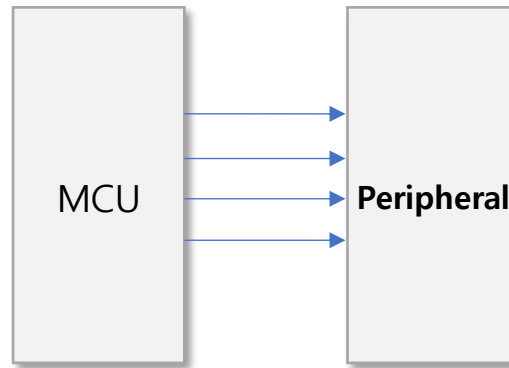
Parallel vs. Serial Data Transfers

8-bit transfer

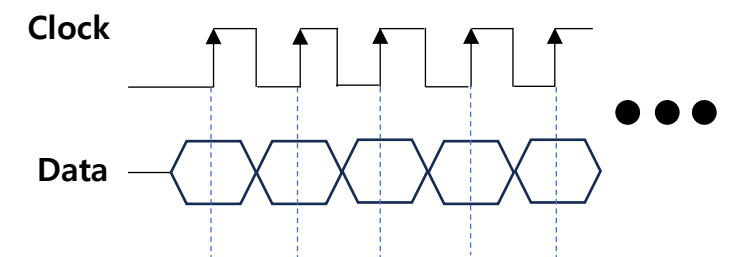
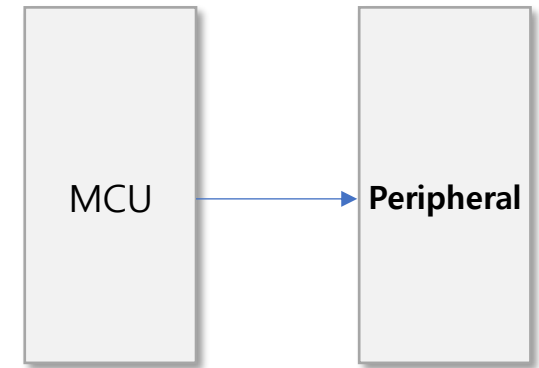


Peripheral에 따라 Clock이 rising edge (0→1) 또는 falling edge (1→0) 일 때 Data를 읽음.

4-bit transfer

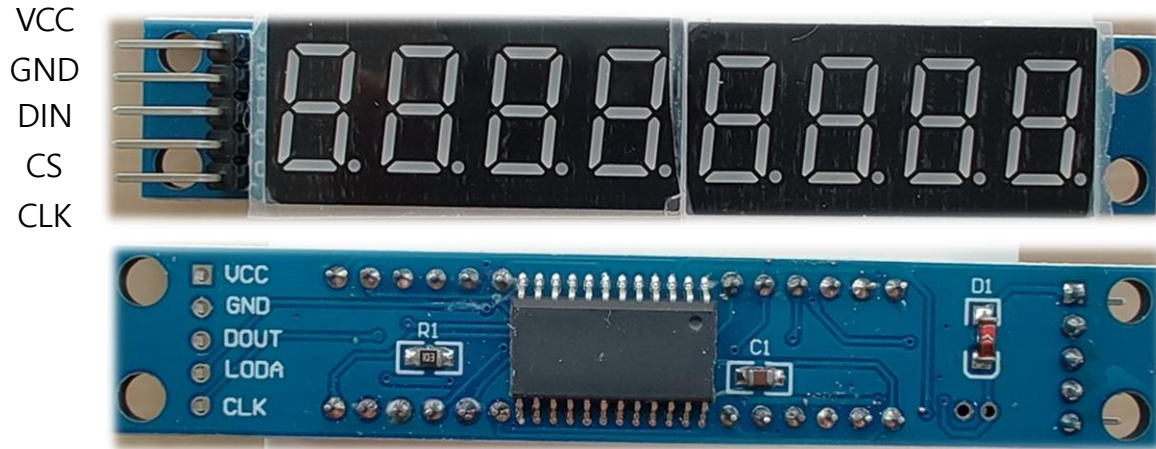


1-bit transfer



여러 방식이 있으며, 그 중 SPI (Serial Peripheral Interface)가 많이 사용됨.

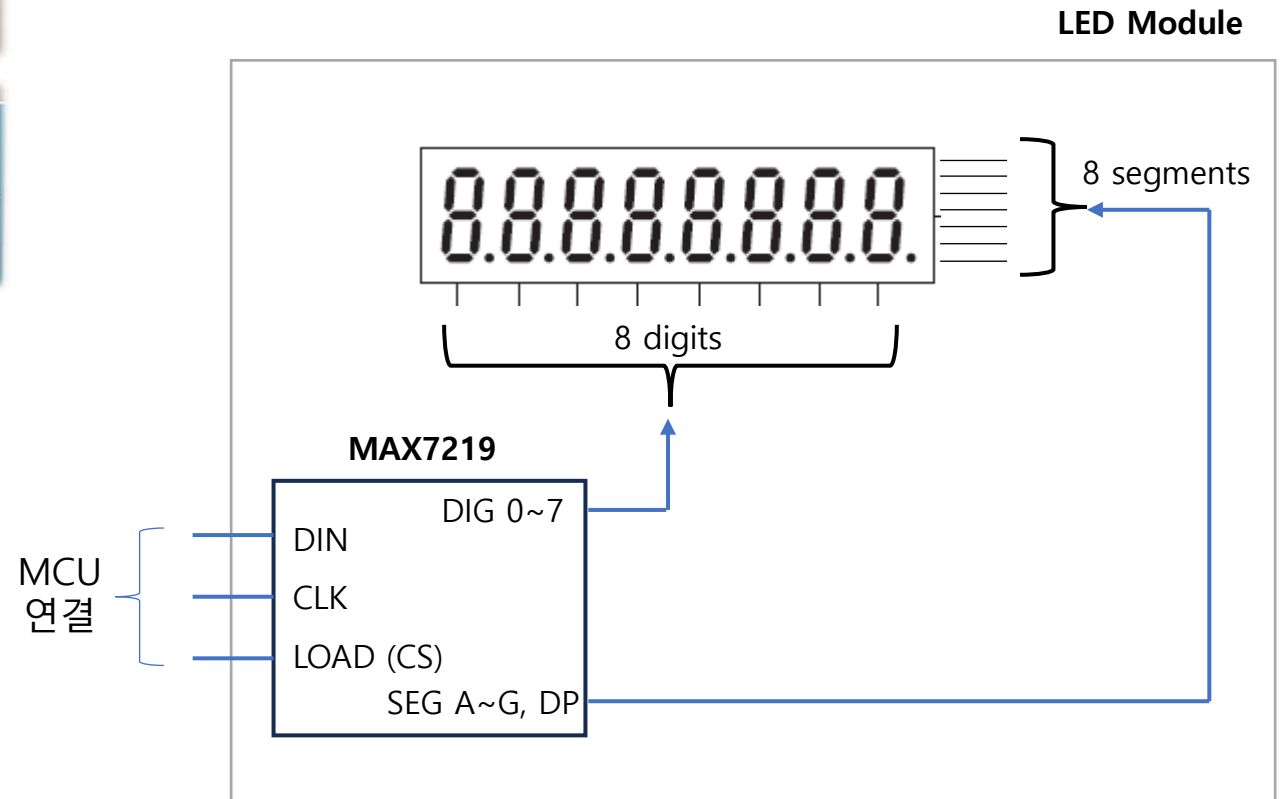
8-Digit 7-Segment LED Module



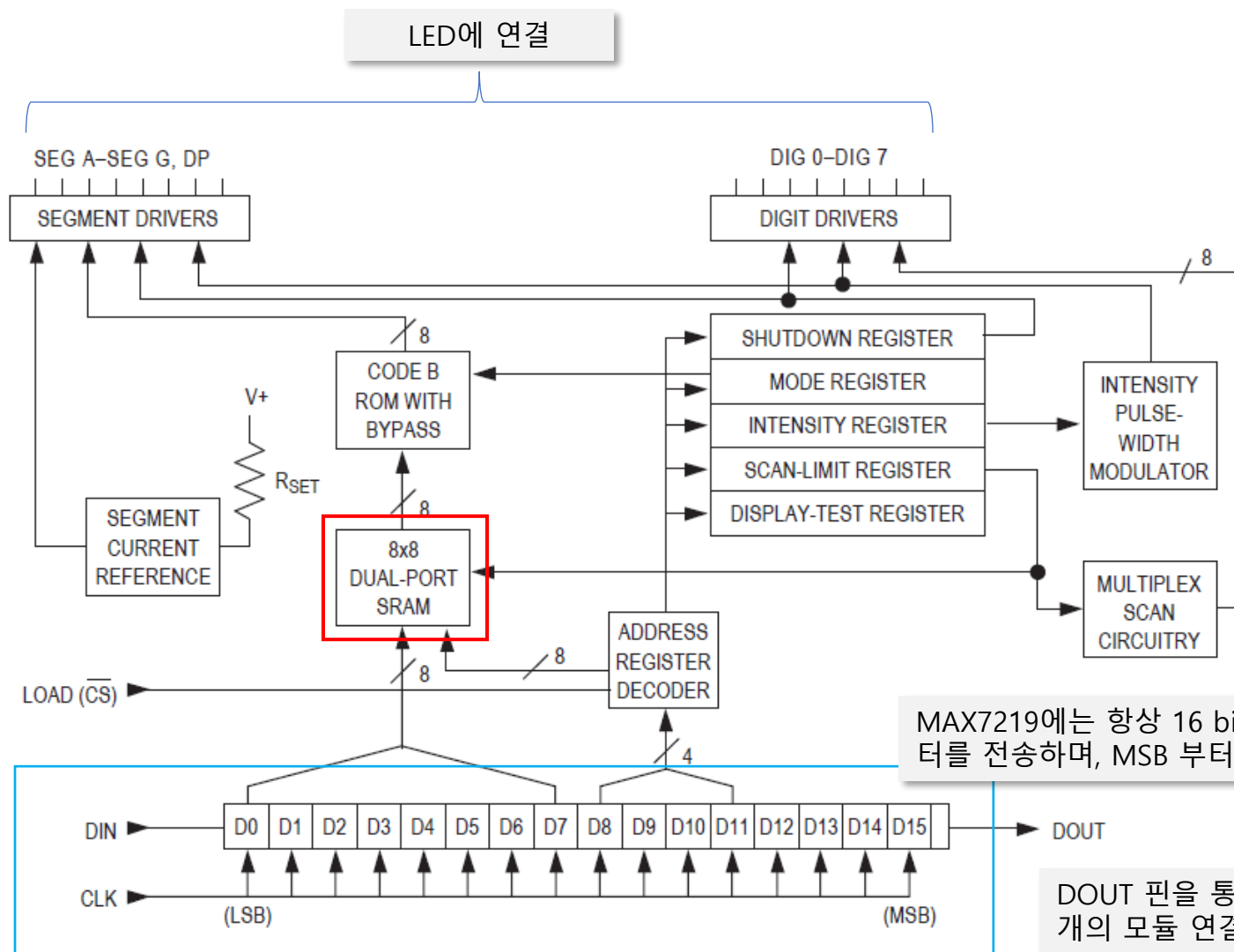
1자리마다 8개, 총 64개의 LED를 3개의 핀 (CLK, DIN, CS)으로 제어.

MAX7219 IC는 SPI를 통해 데이터를 받은 후, 그 데이터를 내부 메모리에 저장하여 64개의 LED를 각각 On/Off 함.

MAX7219 IC는 한 자리 (digit) 마다 8개의 segment 신호를 출력하여 그 자리의 숫자가 순간적으로 표시되게 한 후, 다음 자리의 segment 신호를 출력하는 동작을 반복함 (눈의 잔상 효과를 이용함).



MAX7219 내부 구조

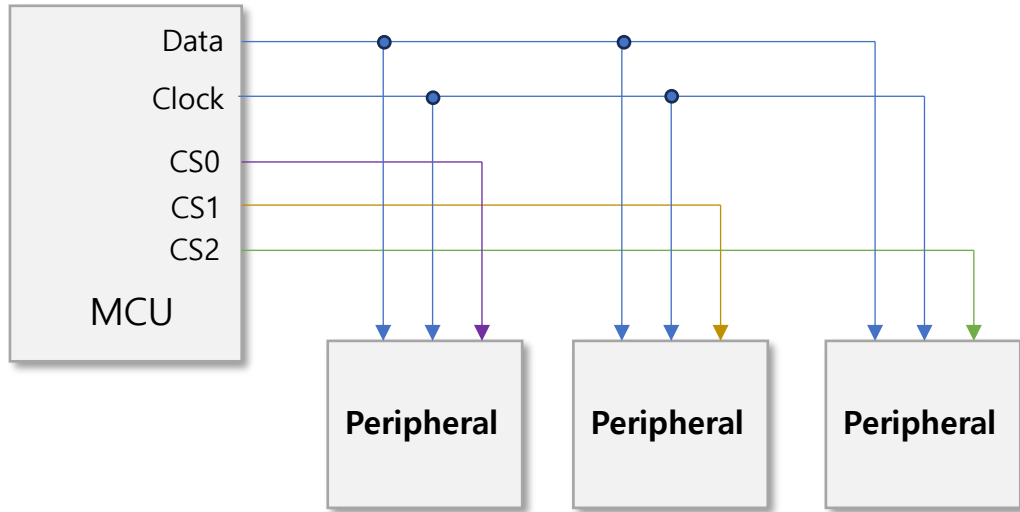


D12~D15	사용 안 함
D8~D11	Register 번호 또는 메모리 주소
D0~D7	Register 또는 메모리에 write할 데이터

MAX7219에는 항상 16 bits의 데이터를 전송하며, MSB 부터 전송함.

DOUT 핀을 통해 여러 개의 모듈 연결도 가능.

Chip Select (CS) 신호



다수의 peripheral에 공통의 Clock 과 Data 신호가 출력 되었을 때, 어느 peripheral이 그 신호를 받을 지를 알려 주는 신호.

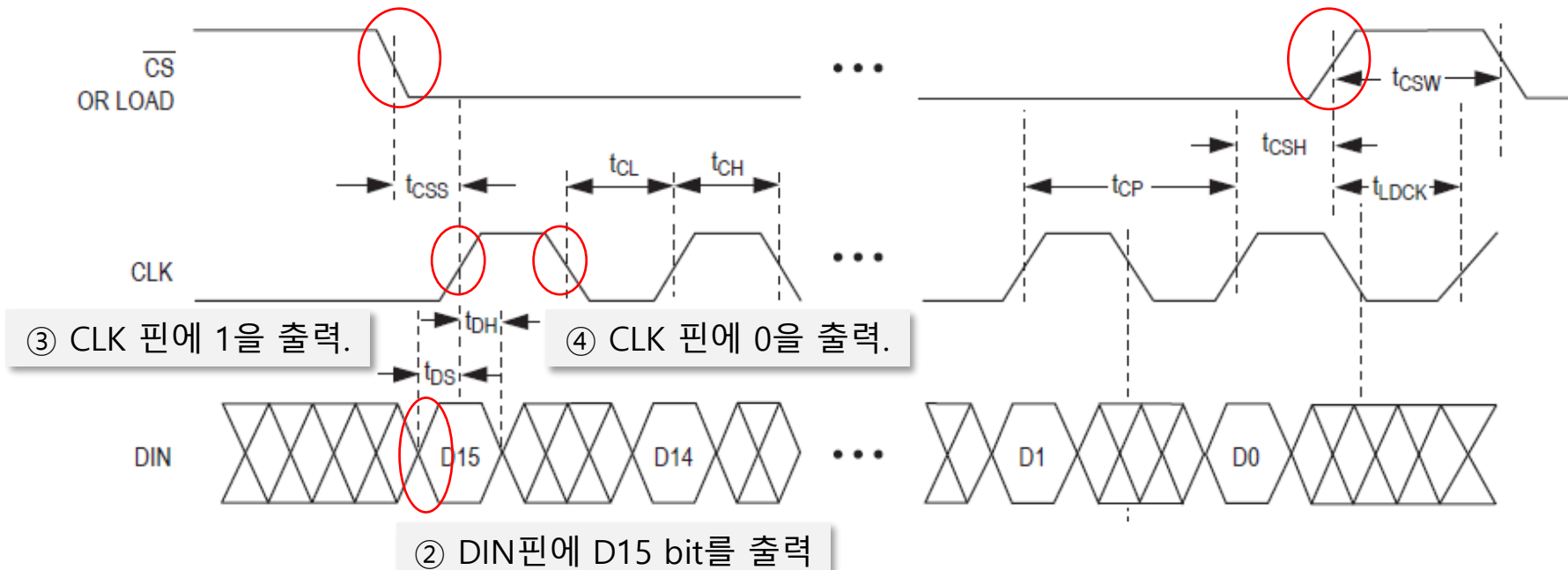
선택되지 않은 peripheral들은 Clock, Data 신호에 대해 반응하지 않음.

CS 신호가 0일 때 활성화 되는 (Active Low) peripheral 들이 많음.

MAX7219 신호 순서와 타이밍

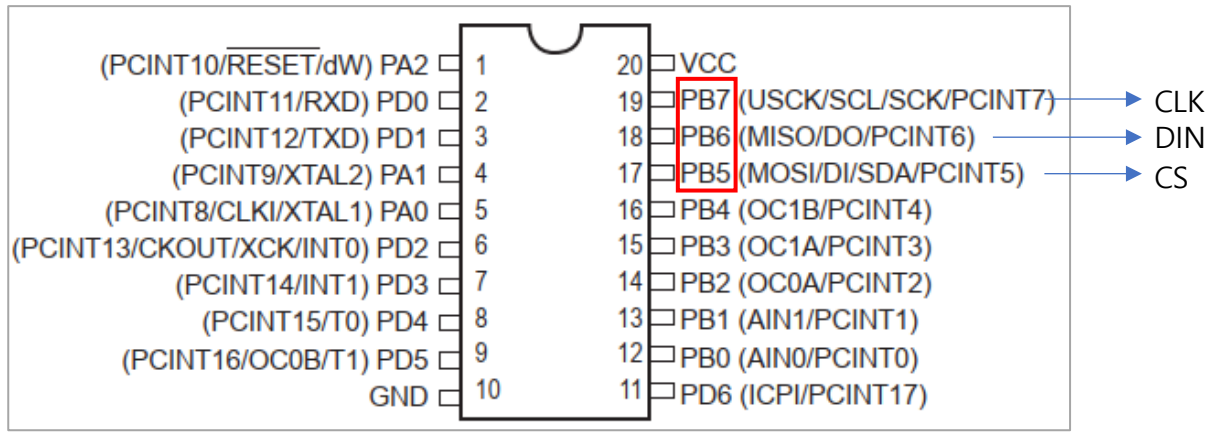
① CS에 0 출력 (전송 시작)

⑥ CS에 1 출력 (전송 종료)



⑤ CLK과 DIN 신호 출력을 16번 반복

ATtiny와 MAX7219 연결



VCC, GND도 연결

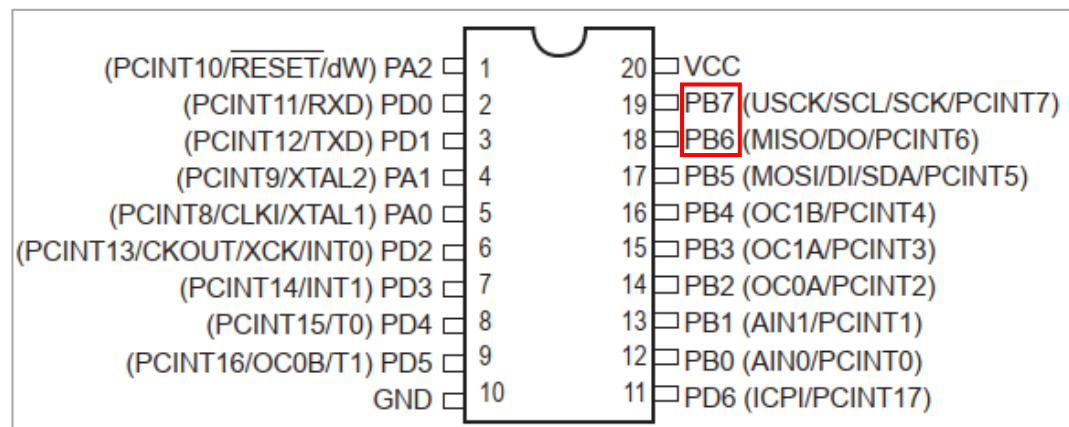
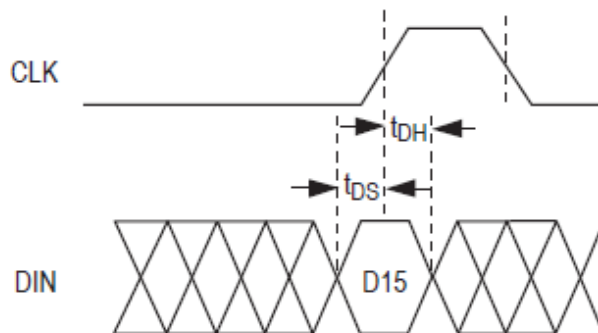
SPI Transmit 함수 (Bit-Bang 방식)

```
#include <avr/io.h>

static void SpiTx (uint8_t data) {
    for (uint8_t i=0; i<8; i++) { // 8번 반복 (8 bits 출력)

        uint8_t bit = (data>>(7-i)) & 1; // MSB부터 차례로 1 bit 를 추출
        PORTB &= ~(1<<6); // PB6 핀에 0 출력
        PORTB |= (bit<<6); // PB6 핀에 bit 값을 출력

        PORTB |= (1<<7); // PB7 핀에 1 출력
        PORTB &= ~(1<<7); // PB7 핀에 0 출력
    }
}
```

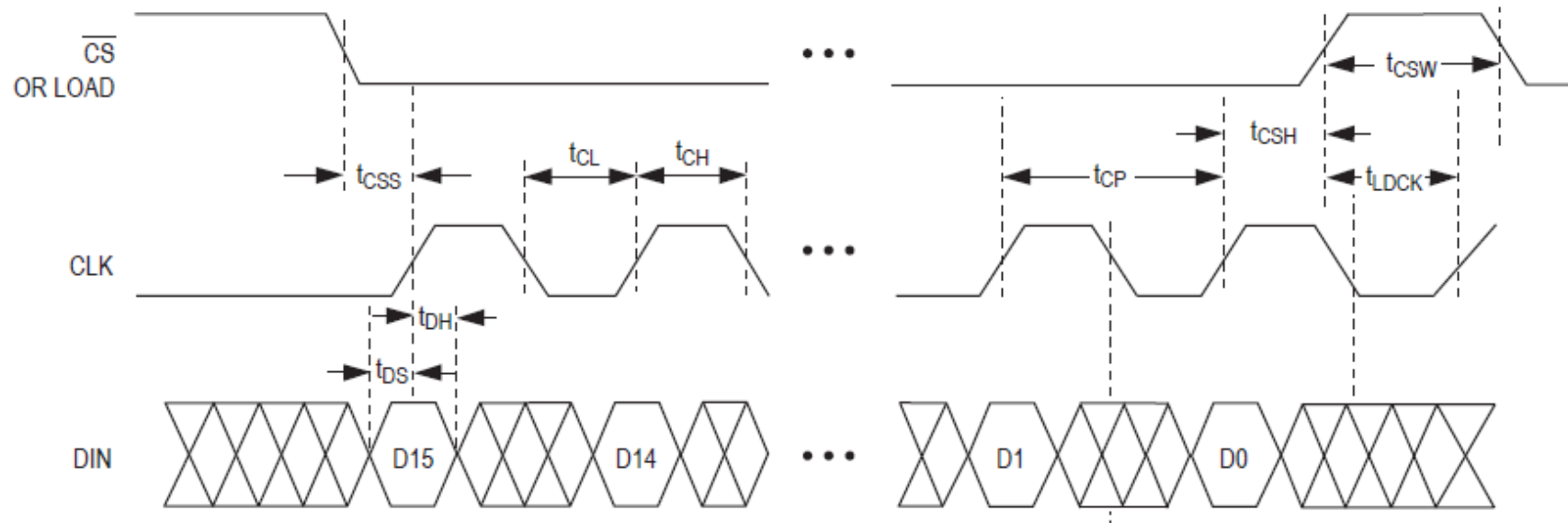


MAX7219 Write 함수 (16 bits 전송)

```
#include <avr/io.h>

#define MAX_CS_DDR    DDRB
#define MAX_CS_PORT    PORTB
#define MAX_CS_BIT    5

static void max7219_write(uint8_t addr, uint8_t data) {
    MAX_CS_PORT &= ~(1<<MAX_CS_BIT); // CS 핀에 0 출력 (전송 시작)
    SpiTx(addr);                       // 8-bit addr 전송
    SpiTx(data);                       // 8-bit data 전송
    MAX_CS_PORT |= (1<<MAX_CS_BIT);   // CS 핀에 1 출력 (전송 종료)
}
```



MAX7219 Registers

Register Name & Address

Address는 8 bits로 전송해야 하지만, 상위 4 bits는 사용 안 함 (X=Don't Care).

Register의 주소 (address)를 먼저 8 bits로 전송하고, 이어서 그 Register에 write 할 값을 8 bits로 전송 (총 16 bits 전송).

Name	D15	D14	D13	D12	D11	D10	D9	D8	HEX
No Op	X	X	X	X	0	0	0	0	0xX0
Digit 0	X	X	X	X	0	0	0	1	0xX1
Digit 1	X	X	X	X	0	0	1	0	0xX2
Digit 2	X	X	X	X	0	0	1	1	0xX3
Digit 3	X	X	X	X	0	1	0	0	0xX4
Digit 4	X	X	X	X	0	1	0	1	0xX5
Digit 5	X	X	X	X	0	1	1	0	0xX6
Digit 6	X	X	X	X	0	1	1	1	0xX7
Digit 7	X	X	X	X	1	0	0	0	0xX8
Decode Mode	X	X	X	X	1	0	0	1	0xX9
Intensity	X	X	X	X	1	0	1	0	0xXA
Scan Limit	X	X	X	X	1	0	1	1	0xXB
Shutdown	X	X	X	X	1	1	0	0	0xXC
Display Test	X	X	X	X	1	1	1	1	0xFF

표시할 LED 데이터를 저장하는 register

동작 설정을 저장하는 register

Decode Mode Register

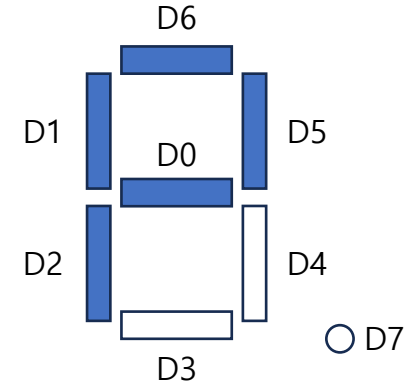
MAX7219는 Decode Mode Register 설정에 따라, Digit 0~7의 각 bit가 직접 LED를 On/Off 할 수도 있고, Digit 0~7의 값을 숫자로 인식할 수도 있음.

[예]

	Register Data							
	D7	D6	D5	D4	D3	D2	D1	D0
Digit 0	0	1	1	0	0	1	1	1

No-Decode Mode

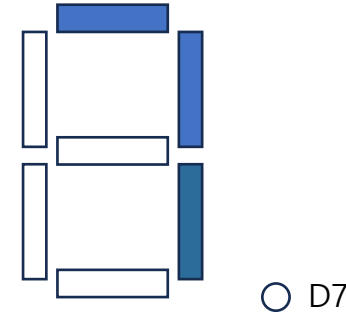
Digit register에 저장된 8-bit 값의 각 bit가 LED의 segment를 직접 on/off 함.



Decode Mode

Digit register에 저장된 8-bit 값 중 하위 4 bits 를 숫자로 인식하여 그 숫자를 표시.

하위 4 bits의 0111은 십진수 7



소수점 (Decimal Point)는 D7에 의해 On/Off 됨

Decode Mode Register

9 이상의 값은 특수 문자를 표시

D7	D6	D5	D4	D3	D2	D1	D0	표시되는 문자
DP	X	X	X	1	0	1	0	-
DP	X	X	X	1	0	1	1	E
DP	X	X	X	1	1	0	0	H
DP	X	X	X	1	1	0	1	L
DP	X	X	X	1	1	1	0	P
DP	X	X	X	1	1	1	1	없음

Decode Mode Register (Address 0x09)

Digit마다 No-Decode 로 사용할 지,
Decode 로 사용할 지 선택할 수 있음.

Bit가 0 이면 그 Digit는 No-Decode
Mode로, 1 이면 Decode Mode로 동작

[예]

Decode Mode	Register Data							
	D7	D6	D5	D4	D3	D2	D1	D0
All digits No-Decode Mode	0	0	0	0	0	0	0	0
Digits 1~7 No-Decode Mode	0	0	0	0	0	0	0	1

● ● ●

Digits 4~7 No-Decode Mode	0	0	0	0	1	1	1	1
---------------------------	---	---	---	---	---	---	---	---

Scan Limit Register



Digit 0부터 시작하여 몇 번 digit까지 표시할지를 선택.

선택하지 않은 상위 digit는 꺼짐.

Scan-Limit Register (Address 0x0B)

Scan Limit	Register Data							
	D7	D6	D5	D4	D3	D2	D1	D0
Digit 0만 표시	X	X	X	X	X	0	0	0
Digit 0~1 표시	X	X	X	X	X	0	0	1
Digit 0~2 표시	X	X	X	X	X	0	1	0
Digit 0~3 표시	X	X	X	X	X	0	1	1
Digit 0~4 표시	X	X	X	X	X	1	0	0
Digit 0~5 표시	X	X	X	X	X	1	0	1
Digit 0~6 표시	X	X	X	X	X	1	1	0
Digit 0~7 표시	X	X	X	X	X	1	1	1

MAX7219 Register 설정 예



- 모든 자리 점등
- Decode Mode
- 최소 밝기
- 01234567 표시

Register	Value	Address	Data
Decode Mode	Decode	0x09	0xFF
Intensity	1/32	0x0A	0x01
Scan Limit	Digits 0~7	0x0B	0x07
Shutdown	Normal Operation	0x0C	0x01
Display Test	Normal Operation	0x0F	0x00
Digit 0	7	0x01	0x07
Digit 1	6	0x02	0x06
Digit 2	5	0x03	0x05
Digit 3	4	0x04	0x04
Digit 4	3	0x05	0x03
Digit 5	2	0x06	0x02
Digit 6	1	0x07	0x01
Digit 7	0	0x08	0x00

Register마다 Address, Data의 2 bytes
를 전송해야 그 Register가 설정됨.

MAX7219 초기화 함수와 예제 코드

Register	Value	Address	Data
Decode Mode	Decode	0x09	0xFF
Intensity	1/32	0x0A	0x01
Scan Limit	Digits 0~7	0x0B	0x07
Shutdown	Normal Operation	0x0C	0x01
Display Test	Normal Operation	0x0F	0x00

```
static void max7219_init(void) {
    const uint8_t initData[] = {
        0x09, 0xff, 0x0a, 0x01, 0x0b, 0x07, 0x0c, 0x01, 0x0f, 0x00 };

    DDRB |= (1<<6); // PB6 핀을 출력 모드로 설정
    DDRB |= (1<<7); // PB7 핀을 출력 모드로 설정
    MAX_CS_DDR |= (1<<MAX_CS_BIT); // CS 핀을 출력 모드로 설정
    MAX_CS_PORT |= (1<<MAX_CS_BIT); // CS 핀에 1 출력

    // initData[] 값을 data와 addr의 pair로 max7219_write 함수에 보냄
    for (unsigned i=0; i<sizeof(initData); i+=2) {
        max7219_write(initData[i], initData[i+1]);
    }
}
```

```
#define F_CPU 8000000

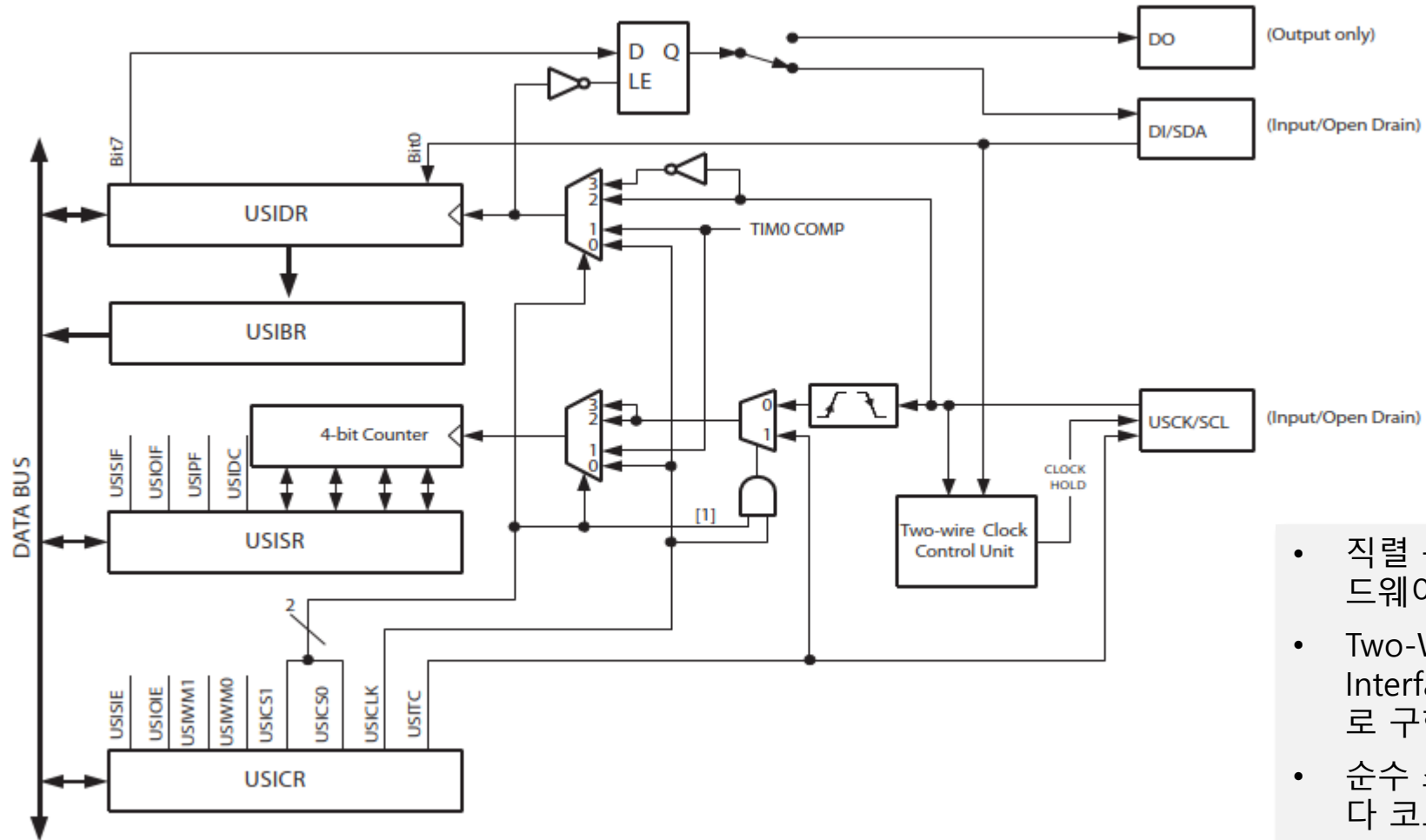
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    max7219_init();
    while (1) {
        for (uint8_t number=0; number<10; number++) {
            for (uint8_t digit=0; digit<8; digit++) {
                max7219_write((7-digit)+1, (number+digit)%10);
            }
            _delay_ms(500);
        }
    }
}
```



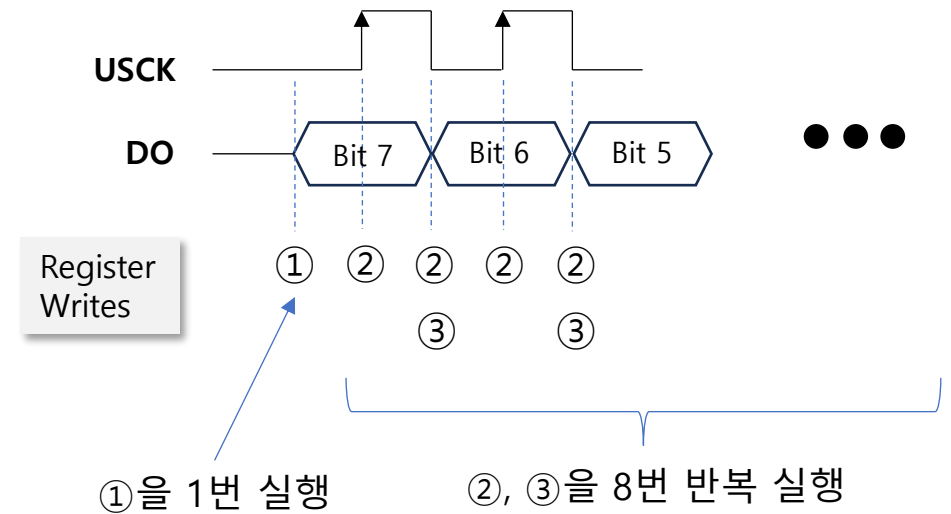
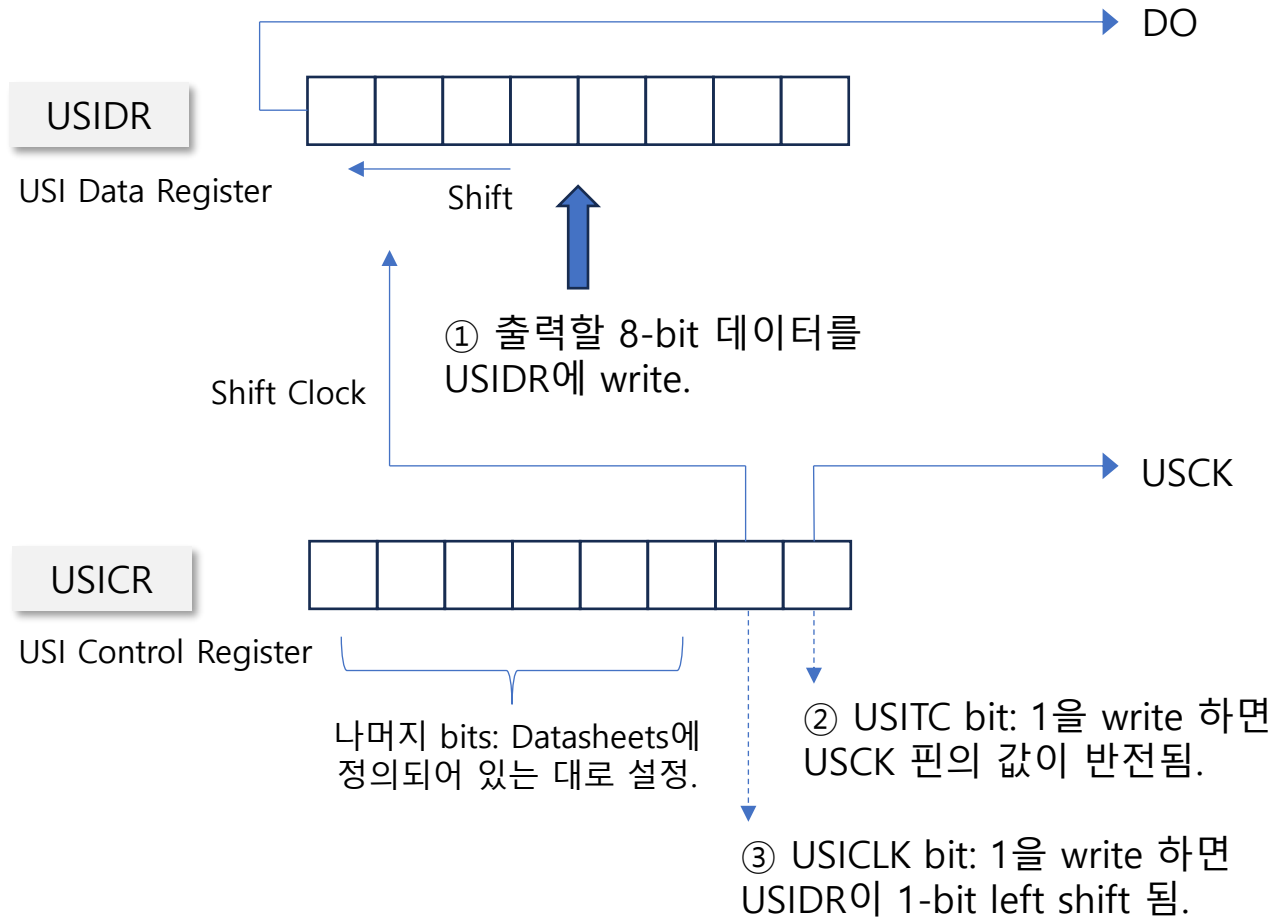
ATtiny의 USI (Universal Serial Interface)

Figure 16-1. Universal Serial Interface, Block Diagram

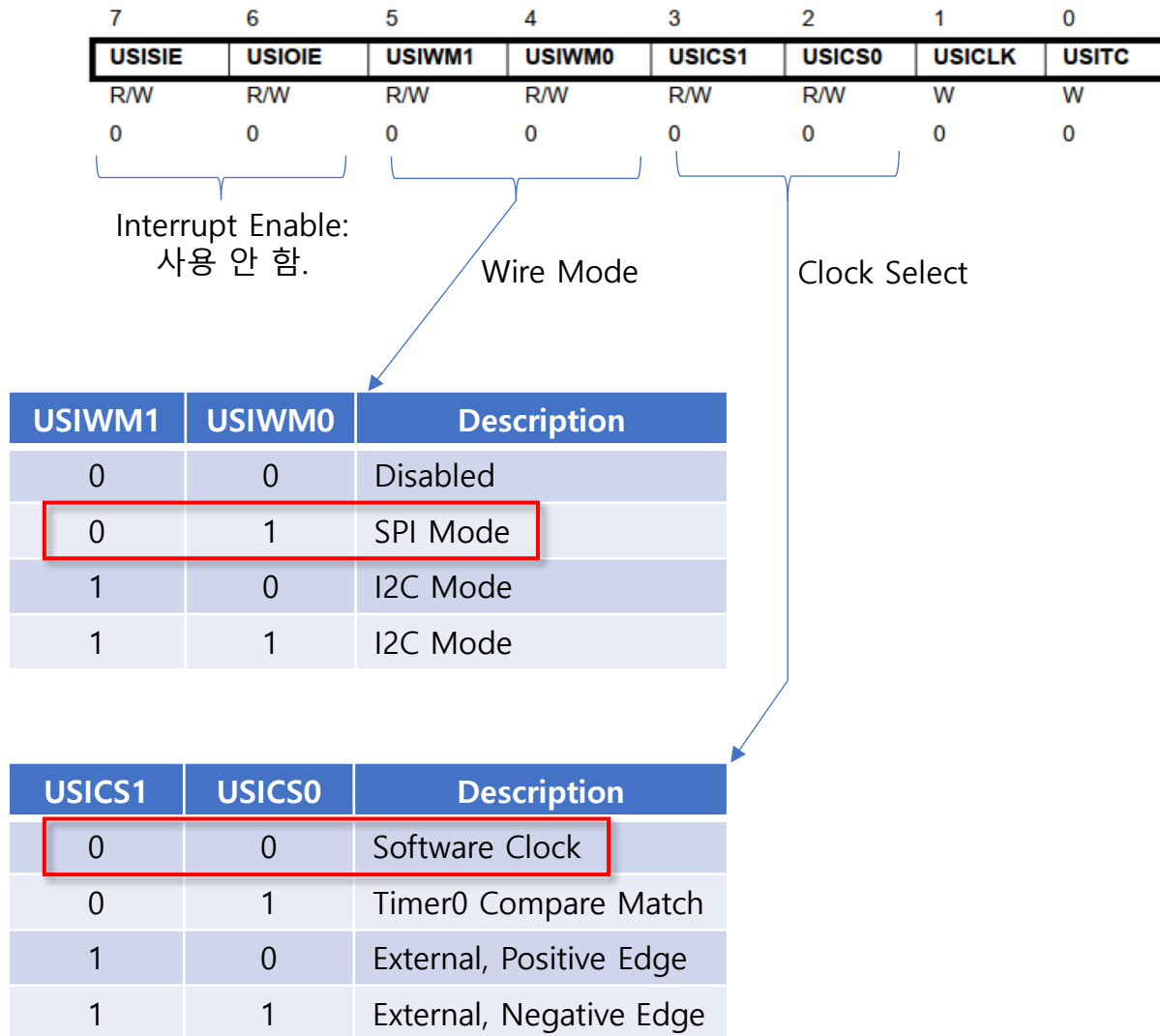


- 직렬 통신을 구현할 수 있는 기본적인 하드웨어.
- Two-Wire Interface (I²C) 및 Three-Wire Interface (SPI) 기능을 최소의 소프트웨어로 구현 가능.
- 순수 소프트웨어 구현 (Bit-Bang) 방식보다 코드가 단순해짐.

USI를 활용한 SPI 출력



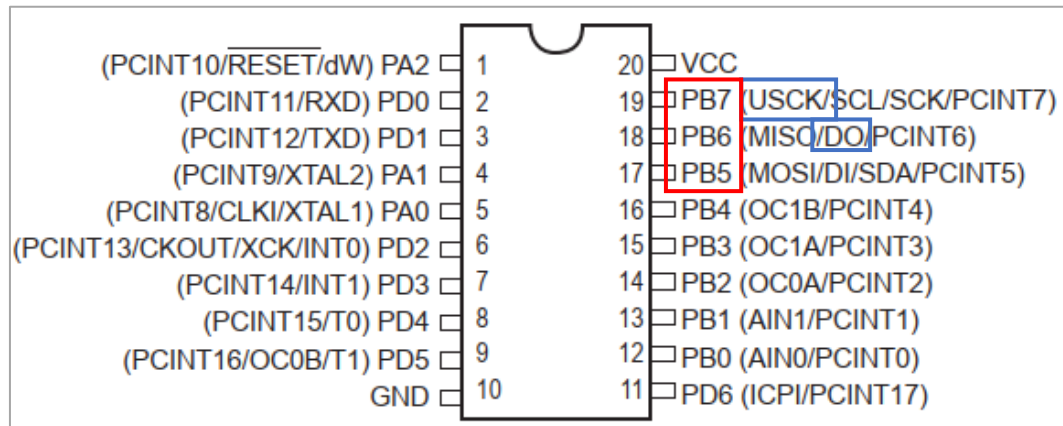
USICR (USI Control Register)



SPI Transmit 함수 (USI 방식)

```
#include <avr/io.h>

static void SpiTx (uint8_t data) {
    USIDR = data;      // 전송할 8-bit data를 USIDR에 저장
    for (uint8_t i=0; i<8; i++) { // 8번 반복
        USICR = ???;    // USIWM=01, USICS=00, USICLK=0, USITC=1
        USICR = ???;    // USIWM=01, USICS=00, USICLK=1, USITC=1
    }
}
```



이전 Bit-Bang 방식에서 PB7 핀을 CLK에, PB6 핀을 DIN에 연결하였으므로, 회로 변경 없이 SpiTx 함수만 대체하면 동일하게 동작함.

Bit-Bang 방식과 USI 방식의 코딩

C Pre-Processor의 #if 조건문을 사용하여 컴파일 때 두 구현 방식 중 하나를 선택.

```
static void SpiTx (uint8_t data) {
    #if 0 // Bit-Bang
        for (uint8_t i=0; i<8; i++) {

            uint8_t bit = (data>>(7-i)) & 1; // MSB부터 차례로 1 bit 를 추출
            PORTB &= ~(1<<6);                // PB6 핀에 0 출력
            PORTB |= (bit<<6);                // PB6 핀에 bit 값을 출력

            PORTB |= (1<<7);                  // PB7 핀에 1 출력
            PORTB &= ~(1<<7);                // PB7 핀에 0 출력
        }
    #else // USI
        USIDR = data;                        // 전송할 8-bit data를 USIDR에 저장
        for (uint8_t i=0; i<8; i++) {
            USICR = ???;                     // USIWM=01, USICS=00, USICLK=0, USITC=1
            USICR = ???;                     // USIWM=01, USICS=00, USICLK=1, USITC=1
        }
    #endif
}
```